

Universidade Presbiteriana Mackenzie

Faculdade de Computação e Informática - FCI

Projeto Prático 2 (LAB2)

Analizador de Desempenho de Cache de CDN com OpenMP

Computação Paralela

Prof. Dr. Jean M. Laine

1. Introdução

1.1 O Cenário: Content Delivery Networks

Sistemas modernos de distribuição de conteúdo dependem fortemente de infraestruturas paralelas e distribuídas para atender milhões de usuários simultaneamente.

Ao acessar plataformas como Netflix, YouTube, Globoplay ou grandes portais de notícias, o usuário normalmente não acessa diretamente um servidor central, mas sim uma *CDN* (*Content Delivery Network*).

O objetivo de uma CDN é manter réplicas de conteúdos distribuídas geograficamente para reduzir:

- latência;
- gargalos;
- custos operacionais;
- tráfego no servidor de origem.

Para isso, sistemas de monitoramento processam continuamente enormes volumes de logs de acesso.

Neste projeto, você desenvolverá um analisador paralelo simplificado capaz de processar milhões de requisições e identificar padrões de acesso em tempo real.

1.2 O Problema da Sincronização

Em cenários reais, determinados conteúdos tornam-se virais e concentram milhões de acessos simultâneos.

Quando múltiplas threads tentam atualizar os mesmos contadores concorrentemente, surgem problemas clássicos de:

- contenção;
- serialização;
- condições de corrida;
- false sharing;
- degradação de cache.

O objetivo deste laboratório é explorar experimentalmente diferentes estratégias de sincronização utilizando OpenMP.

2. Objetivos

Ao final deste projeto, espera-se que o aluno seja capaz de:

- implementar soluções paralelas utilizando OpenMP;
- investigar condições de corrida;
- comparar diferentes granularidades de sincronização;
- analisar impacto de false sharing;
- medir métricas reais de hardware;
- relacionar desempenho de software com arquitetura computacional.

Métricas investigadas:

- tempo de execução;
- speedup;
- IPC (*Instructions Per Cycle*);
- cache misses;
- system time.

3. Arquivos Fornecidos

O script:

```
1 generate_cdn_data.py
```

gera automaticamente os seguintes arquivos:

- manifest.txt
- log_distribuido.txt
- log_concorrente.txt
- gabarito_distribuido.csv
- gabarito_concorrente.csv

Os arquivos são compactados automaticamente em:

```
1 cdn_data_logs.zip
```

3.1 Manifest

O arquivo:

```
1 manifest.txt
```

contém uma URL única por linha.

Exemplo:

```
1 /video/trailer.mp4
2 /images/logo.png
3 /css/main.css
```

Dataset aproximado:

- 100.000 URLs únicas;
- 10 milhões de linhas por log.

3.2 Formato dos Logs

Os logs seguem um formato simplificado inspirado em Apache/Nginx.

Exemplo:

```
1 127.0.0.1 - - [01/Nov/2025:10:00:01 -0300]
2 "GET /video/trailer.mp4 HTTP/1.1" 200 1500
```

Cada linha contém:

- endereço IP;
- timestamp;
- método HTTP;
- URL;

- protocolo HTTP;
- status HTTP;
- quantidade de bytes.

Apenas a URL deve ser extraída e processada.

3.3 Sugestões de Parsing

O objetivo do laboratório NÃO é implementar um parser HTTP complexo.

Sugestões aceitáveis:

- parsing manual simples;
- `sscanf()`;
- `strtok()`.

Evite soluções excessivamente custosas ou bibliotecas externas.

3.4 Distribuição dos Logs

`log_distribuido.txt`

- acessos aproximadamente uniformes;
- baixa contenção.

`log_concorrente.txt`

- aproximadamente 90% das requisições concentram-se em poucas URLs;
- alta contenção;
- simulação de hotspots.

4. Tabela Hash

O projeto utiliza os arquivos:

- `hash_table.h`
- `hash_table.c`

`hash_table.h` contém as declarações.

`hash_table.c` contém a implementação.

A compilação deve ser modular.

Nunca utilize `#include` de arquivos `.c`.

4.1 Funções Disponíveis

```
1 HashTable* ht_create(size_t size);
2
3 void ht_insert(HashTable* ht,
4               const char* url);
5
6 CacheNode* ht_get(HashTable* ht,
7                  const char* url);
8
9 void ht_save_results(HashTable* ht,
10                    const char* filename);
```

4.2 Modelo de Concorrência

A tabela hash deve ser construída inicialmente de forma sequencial.

Após a construção:

- nenhuma thread poderá inserir novos elementos;
- nenhuma thread poderá remover elementos;
- apenas o campo `hit_count` poderá ser atualizado concorrentemente.

Assim, chamadas simultâneas para:

```
1 ht_get()
```

são consideradas válidas durante a execução paralela.

Todas as URLs presentes nos logs estão garantidamente presentes em:

```
1 manifest.txt
```

Portanto, durante a execução normal, `ht_get()` não deverá retornar NULL.

4.3 Tamanho da Tabela Hash

Sugestão:

utilizar quantidade de buckets significativamente maior que o número de hotspots.

Exemplo razoável:

```
1 131071
```

(primo próximo de potência de 2).

5. Descrição do Problema

5.1 Fase 1: Construção da Estrutura

Leia:

```
1 manifest.txt
```

e construa a tabela hash.

Cada URL deve iniciar com:

```
1 hit_count = 0
```

5.2 Fase 2: Processamento dos Logs

O programa deverá processar:

- log_distribuido.txt
- log_concorrente.txt

Pipeline esperado:

1. abrir arquivo;
2. ler linhas;
3. extrair URL;
4. localizar URL na hash;
5. incrementar contador;
6. salvar resultados.

5.3 Sugestão de Paralelização

Uma estratégia recomendada é:

- carregar linhas do log em memória;
- armazenar ponteiros em um vetor;
- utilizar `#pragma omp parallel for`.

Outras estratégias também são permitidas.

6. Versões Obrigatórias

Executáveis esperados:

- analyzer_seq
- analyzer_par_atomic
- analyzer_par_critical
- analyzer_par_lock

6.1 Sequencial

Implementação em thread única.

Baseline de corretude e desempenho.

6.2 OpenMP Critical

Utilize:

```
1 #pragma omp critical
```

Granularidade grossa.

6.3 OpenMP Atomic

Utilize:

```
1 #pragma omp atomic update
```

Sincronização baseada em instruções atômicas de hardware.

6.4 OpenMP Bucket Lock

Nesta versão, a sincronização deve ser realizada utilizando **locks independentes para cada bucket da tabela hash**.

Essa abordagem representa uma estratégia de sincronização de **granularidade média**, buscando equilibrar:

- segurança de concorrência;
- redução de contenção;
- paralelismo;
- custo de sincronização.

O Problema do Lock Global

Na versão utilizando:

```
1 #pragma omp critical
```

todas as threads disputam a mesma região crítica.

Isso significa que:

- apenas uma thread pode atualizar qualquer contador por vez;
- o paralelismo efetivo é reduzido;
- ocorre forte serialização;
- o custo de espera cresce rapidamente com o número de threads.

Mesmo que duas threads estejam acessando URLs completamente diferentes, elas continuarão bloqueando umas às outras.

Em cenários com milhões de acessos simultâneos, essa estratégia torna-se extremamente limitada.

A Ideia do Bucket Lock

A tabela hash é composta por múltiplos buckets.

Cada bucket armazena um subconjunto das URLs.

A ideia do *Bucket Locking* é simples:

- cada bucket possui seu próprio lock;
- threads só competem entre si se acessarem o mesmo bucket;
- buckets diferentes podem ser acessados simultaneamente.

Assim, o paralelismo aumenta significativamente.

Exemplo conceitual:

```
1 Bucket 0 -> Lock 0
2 Bucket 1 -> Lock 1
3 Bucket 2 -> Lock 2
4 ...
5 Bucket N -> Lock N
```

Se duas threads acessarem buckets distintos, não haverá bloqueio entre elas.

Fluxo Esperado

O fluxo típico de execução é:

1. calcular hash da URL;
2. descobrir bucket correspondente;
3. adquirir lock daquele bucket;
4. localizar elemento na lista encadeada;
5. atualizar `hitcount`;
6. liberar lock.

Exemplo Conceitual

```
1 size_t bucket =  
2 hash_function(url) % table_size;  
3  
4 omp_set_lock(&locks[bucket]);  
5  
6 node->hit_count++;  
7  
8 omp_unset_lock(&locks[bucket]);
```

Estrutura Esperada

Sugestão:

- criar um array de:

```
1 omp_lock_t
```

- utilizar exatamente a mesma quantidade de locks da quantidade de buckets;
- inicializar todos os locks antes da execução paralela;
- destruir todos os locks ao final da execução.

Inicialização dos Locks

Exemplo:

```
1 omp_lock_t* locks =  
2 malloc(sizeof(omp_lock_t) * table_size);  
3  
4 for (size_t i = 0;  
5      i < table_size;
```

```
6     i++) {  
7  
8     omp_init_lock(&locks[i]);  
9 }
```

Destruição dos Locks

```
1 for (size_t i = 0;  
2     i < table_size;  
3     i++) {  
4  
5     omp_destroy_lock(&locks[i]);  
6 }  
7  
8 free(locks);
```

Granularidade da Sincronização

Essa abordagem possui granularidade intermediária:

- mais paralela que:

```
1 #pragma omp critical
```

- potencialmente mais barata que múltiplos atomics em cenários específicos;
- menos escalável que estruturas lock-free avançadas, porém muito mais simples de implementar.

Vantagens

- reduz contenção global;
- permite paralelismo real entre buckets distintos;
- diminui tempo de espera;
- melhora escalabilidade;
- aproxima-se mais de arquiteturas reais de servidores.

Desvantagens

- maior consumo de memória;
- custo de inicialização/destruição dos locks;
- possibilidade de hotspots em buckets muito acessados;
- pior localidade de cache dependendo da implementação.

Hotspots e Contenção

Mesmo utilizando bucket locks, a contenção ainda pode existir.

Se milhares de threads acessarem URLs mapeadas para o mesmo bucket:

- ocorrerá disputa pelo mesmo lock;
- o bucket se tornará um hotspot;
- o paralelismo local será reduzido.

Por esse motivo, a escolha da função hash e do tamanho da tabela influencia diretamente o desempenho do sistema.

Relação com Sistemas Reais

Estratégias semelhantes são utilizadas em:

- caches concorrentes;
- servidores web;
- bancos de dados;
- sistemas de armazenamento distribuído;
- tabelas hash concorrentes de alto desempenho.

O objetivo desta etapa é compreender como a granularidade da sincronização afeta:

- escalabilidade;
- throughput;
- utilização dos núcleos;
- eficiência paralela.

7. Verificação de Corretude

Cada implementação deverá gerar:

```
1 results.csv
```

Utilize:

```
1 ht_save_results(ht, "results.csv");
```

7.1 Validação

```
1 sort results.csv > sorted_res.csv
2 sort gabarito_distribuido.csv > sorted_gab.csv
3
4 diff -s sorted_res.csv sorted_gab.csv
```

A validação será considerada correta apenas se os arquivos forem idênticos.

8. Experimentos

O projeto deve ser executado preferencialmente em Linux.

Recomenda-se:

- GCC 11 ou superior;
- máquina com pelo menos 4 núcleos físicos.

8.1 Compilação

```
1 gcc -O2 -fopenmp analyzer_par_atomic.c \
2     hash_table.c -o analyzer_par_atomic
```

8.2 Threads

```
1 export OMP_NUM_THREADS=8
```

8.2.1 Ferramentas de Profiling e Análise

Durante este laboratório, serão utilizadas ferramentas de análise de desempenho para investigar o comportamento das implementações paralelas.

As principais ferramentas sugeridas são:

- **perf**
- **Valgrind/Cachegrind**

Embora ambas permitam investigar desempenho, elas possuem objetivos e metodologias diferentes.

perf

O **perf** é uma ferramenta de profiling baseada em contadores reais de hardware. Ela permite medir eventos como:

- instruções executadas;
- ciclos de CPU;

- cache misses;
- branch mispredictions;
- page faults.

Exemplo:

```
1 perf stat -e instructions,cycles \
2 ./analyzer_par_atomic \
3 log_concorrente.txt
```

Exemplo simplificado de saída:

```
1 8,214,551,020 instructions
2 2,105,331,442 cycles
3
4 IPC = 3.90
```

Valgrind e Cachegrind

O Valgrind é uma suíte de ferramentas de análise dinâmica.

Neste projeto, a ferramenta mais relevante é o:

```
1 Cachegrind
```

Exemplo:

```
1 valgrind --tool=cachegrind \
2 ./analyzer_par_atomic \
3 log_concorrente.txt
```

Exemplo simplificado de saída:

```
1 D refs:      5,201,553
2 D1 misses:   412,330
3 LL misses:   21,442
```

Diferenças Entre perf e Cachegrind

Característica	perf	Cachegrind
Fonte dos dados	Hardware real	Simulação
Precisão temporal	Muito alta	Moderada
Impacto na execução	Baixo overhead	Alto overhead
Medição de IPC	Sim	Não diretamente

8.3 Experimento A: Escalabilidade

Dataset:

```
1 log_distribuido.txt
```

Executar com:

- 1 thread;
- 2 threads;
- 4 threads;
- 8 threads.

Exemplo:

```
1 time ./analyzer_par_atomic log_distribuido.txt
```

Produzir:

- tabela de tempos;
- gráfico de speedup.

8.4 Speedup

$$Speedup = \frac{T_{sequencial}}{T_{paralelo}}$$

8.5 Experimento B: Contenção

Dataset:

```
1 log_concorrente.txt
```

System Time

Execute:

```
1 /usr/bin/time -v \  
2 ./analyzer_par_critical \  
3 log_concorrente.txt
```

Exemplo simplificado de saída:

```
1 User time (seconds): 18.20  
2 System time (seconds): 12.55  
3 Percent of CPU this job got: 780%  
4 Elapsed (wall clock) time: 0:04.01
```

Analisar:

- crescimento do `System time`;
- impacto da sincronização excessiva;
- overhead do sistema operacional.

IPC

Execute:

```
1 perf stat -e instructions,cycles \  
2 ./analyzer_par_atomic \  
3 log_concorrente.txt
```

Exemplo simplificado de saída:

```
1 8,214,551,020 instructions  
2 2,105,331,442 cycles
```

$$IPC = \frac{instructions}{cycles}$$

8.6 Experimento C: False Sharing

Modificar:

```
1 typedef struct CacheNode {  
2     char* url;  
3     long hit_count;  
4     struct CacheNode* next;  
5     long padding[5];  
6 } CacheNode;
```

Versão esperada:

```
1 analyzer_par_atomic_padded
```

Medição

Execute:

```
1 perf stat -e cache-references,cache-misses \  
2 ./analyzer_par_atomic_padded \  
3 log_concorrente.txt
```

Exemplo simplificado de saída:

```
1 cache-references: 4,521,991  
2 cache-misses:    221,552
```

Opcionalmente:

```
1 valgrind --tool=cachegrind \  
2 ./analyzer_par_atomic_padded \  
3 log_concorrente.txt
```

9. Questões de Análise

- Q1:** Por que a versão critical pode degradar sob alta contenção?
- Q2:** Como locks por bucket equilibram sincronização e paralelismo?
- Q3:** Compare cache misses entre versões atomic e padded.
- Q4:** Houve speedup superlinear?
- Q5:** Quando o aumento de memória causado pelo padding torna-se vantajoso?
- Q6:** Qual solução seria utilizada em uma CDN real?
- Q7:** Como listas encadeadas impactam localidade de cache?

10. Estrutura do Projeto

```
1 projeto/  
2 |-- analyzer_seq.c  
3 |-- analyzer_par_atomic.c  
4 |-- analyzer_par_critical.c  
5 |-- analyzer_par_lock.c  
6 |-- hash_table.c  
7 |-- hash_table.h  
8 |-- Makefile  
9 |-- manifest.txt  
10 |-- log_distribuido.txt  
11 |-- log_concorrente.txt  
12 '-- results.csv
```

A organização correta do projeto facilita a compilação, manutenção e execução dos experimentos.

Projetos mal organizados tornam mais difícil:

- reproduzir experimentos;
- automatizar compilação;
- depurar problemas;
- medir desempenho corretamente.

Por esse motivo, o uso de:

```
1 Makefile
```

é obrigatório.

10.1 O que é um Makefile?

O *Makefile* é um arquivo utilizado pela ferramenta:

```
1 make
```

para automatizar compilação de programas.

Em projetos reais de sistemas paralelos, HPC (*High Performance Computing*) e software científico, o uso de Makefiles é extremamente comum.

O Makefile permite:

- compilar múltiplos arquivos automaticamente;
- evitar recompilações desnecessárias;
- padronizar flags de compilação;
- reduzir erros manuais;
- facilitar execução em diferentes máquinas.

10.2 Compilação Manual vs Automatizada

Sem Makefile, seria necessário executar comandos como:

```
1 gcc -O2 -fopenmp analyzer_seq.c \  
2     hash_table.c -o analyzer_seq  
3  
4 gcc -O2 -fopenmp analyzer_par_atomic.c \  
5     hash_table.c -o analyzer_par_atomic  
6  
7 gcc -O2 -fopenmp analyzer_par_critical.c \  
8     hash_table.c -o analyzer_par_critical
```

Isso rapidamente se torna difícil de manter.

Com Makefile, basta executar:

```
1 make
```

e todas as versões serão compiladas automaticamente.

10.3 Exemplo de Makefile

```
1 CC=gcc  
2  
3 CFLAGS=-O2 -fopenmp  
4  
5 TARGETS=analyzer_seq \  
6         analyzer_par_atomic \  
7         analyzer_par_critical \  
8         analyzer_par_lock
```

```
9
10 all: $(TARGETS)
11
12 analyzer_seq:
13     $(CC) $(CFLAGS) \
14     analyzer_seq.c hash_table.c \
15     -o analyzer_seq
16
17 analyzer_par_atomic:
18     $(CC) $(CFLAGS) \
19     analyzer_par_atomic.c hash_table.c \
20     -o analyzer_par_atomic
21
22 analyzer_par_critical:
23     $(CC) $(CFLAGS) \
24     analyzer_par_critical.c hash_table.c \
25     -o analyzer_par_critical
26
27 analyzer_par_lock:
28     $(CC) $(CFLAGS) \
29     analyzer_par_lock.c hash_table.c \
30     -o analyzer_par_lock
31
32 clean:
33     rm -f $(TARGETS)
```

10.4 Explicação do Makefile

- **CC=gcc**
Define o compilador utilizado.
- **CFLAGS=-O2 -fopenmp**
Define flags de compilação:
 - **-O2**: otimização;
 - **-fopenmp**: habilita OpenMP.
- **all**
Compila todos os executáveis.
- **clean**
Remove arquivos compilados.

10.5 Como Executar

Compilar:

```
1 make
```

Limpar executáveis:

```
1 make clean
```

Executar programa:

```
1 ./analyzer_par_atomic \
2 log_distribuido.txt
```

10.6 Erros Comuns

- esquecer `-fopenmp`;
- usar `#include "arquivo.c"`;
- misturar arquivos objeto antigos;
- esquecer dependências;
- utilizar TAB incorretamente no Makefile.

IMPORTANTE:

em Makefiles, comandos devem iniciar com TAB e não com espaços.

10.7 Qualidade de Engenharia

Projetos de computação paralela frequentemente envolvem:

- múltiplos executáveis;
- benchmarks;
- profiling;
- scripts automatizados;
- experimentos reproduzíveis.

Por esse motivo, organização e automação fazem parte da avaliação técnica do projeto.

11. Relatório

O relatório deve conter:

- metodologia;
- ambiente experimental;

- tabelas;
- gráficos;
- análise crítica;
- conclusão.

12. Avaliação

Critério	Descrição	Peso
Corretude e Funcionalidade	Código compila corretamente e gera resultados idênticos.	3,0
Implementação Paralela	Uso correto das estratégias de sincronização.	4,0
Relatório e Análise	Qualidade da análise experimental.	3,0

13. Regulamento

Fraude acadêmica implica nota zero.

Todos os integrantes deverão participar da apresentação e defesa.

14. Grupos

Grupos de:

- 3 alunos;
- 4 alunos.